

THE ESSENTIAL GUIDE TO DUCKLAKE

ARCHITECTURE, OPERATIONS, AND ECOSYSTEM

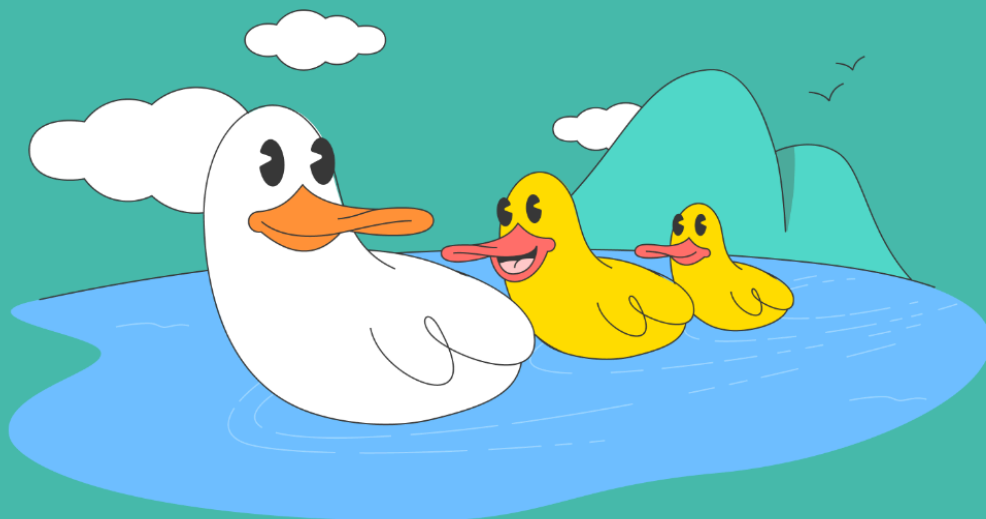


Table of Contents

- Part I: Foundational Concepts and Quick Start..... 4
 - The DuckLake Architecture: A Database-First Approach to the Lakehouse..... 4
 - Hands-on Tutorial: Building Your First DuckLake..... 8
- Part II: Technical Reference and Operations.....12
 - Complete SQL Reference and API Documentation..... 12
 - Performance Tuning & Best Practices..... 15
 - Operations & Monitoring in Production.....18
- Part III: Integration, Migration, and Security..... 22
 - The Integration Cookbook: DuckLake in the Modern Data Stack..... 22
 - The Migration Playbook: Moving to DuckLake..... 24
 - Security Implementation Guide..... 27
- Part IV: Ecosystem, Context, and Community.....30
 - Cost Analysis & Optimization..... 30
 - Limitations, Trade-offs, and Future Direction..... 32
 - Developer & Community Resources..... 34
 - Conclusion..... 35

Part I: Foundational Concepts and Quick Start

This part establishes the core architectural principles of DuckLake, contrasting it with existing formats and providing a practical, hands-on entry point for new users.

The DuckLake Architecture: A Database-First Approach to the Lakehouse

DuckLake represents a fundamental rethinking of the lakehouse paradigm. Instead of building database-like features on top of a file system, DuckLake returns to first principles, moving the complex task of metadata management back to its native domain: a transactional SQL database. This architectural decision simplifies the entire system, enhances performance, and leverages decades of proven database technology to solve the modern data lake's most pressing challenges.

Reimagining the Lakehouse: Moving Beyond File-Based Metadata

The evolution of data lakes into lakehouses was driven by the need to bring database-like reliability, such as ACID transactions and schema evolution, to data stored in open formats on cloud object storage. However, first-generation lakehouse formats like Apache Iceberg and Delta Lake were designed with a core constraint: avoiding a dependency on a traditional database for metadata management.¹

This constraint led to the development of complex, file-based metadata systems. Table state, schema history, and transaction logs are managed through a hierarchy of files, often a "maze of JSON and Avro files", stored alongside the data in object storage.¹ While functional, this approach introduces significant incidental complexity and performance bottlenecks.¹ Every query or write operation may require multiple network round trips to discover, download, and parse various metadata files (e.g., metadata.json, manifest lists, manifest files) before any data can be accessed.² Furthermore, this architecture is susceptible to the "small file problem," where frequent, small updates generate a "forest of tiny files", creating substantial operational overhead for compaction and garbage collection.²

DuckLake's architecture is predicated on a rejection of this initial constraint. It is built on two simple truths:

1. Storing data in open, columnar formats like Parquet on scalable blob storage is a powerful and flexible approach that prevents vendor lock-in.¹
2. Managing complex, transactional metadata is a problem that database management systems have been optimized to solve for decades.¹

By embracing this, DuckLake proposes an elegantly simple architecture: it moves all metadata structures for both the catalog and individual tables into a standard SQL database, while the data remains in Parquet files on object storage.² Every operation, from a simple INSERT to a complex ALTER TABLE, becomes a pure-SQL transaction against the catalog database.¹ This design leverages the database's native, battle-hardened ACID guarantees to provide true multi-table transactions, a capability that remains a significant challenge for single-table-oriented, file-based formats.³ This philosophical shift from reinventing database features on a file system to leveraging an actual database results in a system that is simpler to operate, more performant for many workloads, and more accessible to teams with existing database expertise.

Core Components: The SQL Catalog and the Parquet Data Store

The DuckLake architecture is composed of two primary, decoupled components: a metadata catalog and a data store. This separation of concerns allows for independent scaling and management of metadata, compute, and storage.

The Catalog Database

The catalog database is the transactional heart of a DuckLake. It stores all metadata required to define the lakehouse, including schemas, table definitions, partition information, column statistics, and the complete history of snapshots.

- **Requirements:** Any SQL database that provides standard ACID transaction support and enforces primary key constraints is suitable for use as a DuckLake catalog.¹ The internal schema used by DuckLake is intentionally kept simple to maximize compatibility.¹
- **Supported Systems:** Common choices for the catalog backend include PostgreSQL for enterprise-scale deployments, MySQL for its mature ecosystem, SQLite for multi-client local use, and DuckDB itself for local development.⁷
- **Operational Simplicity:** A significant advantage of this approach is that it relies on familiar, commoditized technology. Most organizations already possess deep operational expertise in managing relational databases like PostgreSQL, which greatly simplifies deployment and reduces the learning curve compared to managing a bespoke catalog service.¹

The Data Store

The physical data in a DuckLake is stored as immutable Parquet files in a cloud-native object store. This adheres to the best practices of modern data lakes, ensuring data portability, open access, and cost-effective scalability.

- **Format:** Data is stored in the open, columnar Apache Parquet format, which

provides excellent compression and encoding optimizations for analytical workloads.³ Using Parquet ensures that the data can be read by a wide variety of query engines and tools, preventing vendor lock-in.¹

- **Storage Systems:** DuckLake is compatible with any major object storage system, including Amazon S3, Google Cloud Storage (GCS), Azure Blob Storage, and Cloudflare R2, as well as local file systems for development.¹
- **Immutability:** Data files in DuckLake are immutable. Operations like updates or deletes do not modify existing files in place; instead, they create new data files and/or delete files, with the changes being recorded atomically in the catalog database.¹ Subsequent maintenance activities create new files and can also delete any de-referenced parquet files as well as related metadata.

Architectural Comparison: DuckLake vs. Iceberg and Delta Lake

While all three formats aim to provide a reliable data lakehouse, DuckLake's database-centric architecture leads to fundamental differences in how they handle core operations.

- **Metadata Operations:** In Iceberg and Delta Lake, discovering the current state of a table requires a client to traverse a file-based hierarchy. This often involves multiple sequential network requests: fetching the root metadata file, then a manifest list, then one or more manifest files to get the list of data files.² In DuckLake, this entire process is replaced by a single SQL query to the catalog database. The database can leverage indexes to perform highly efficient metadata lookups and use standard WHERE clauses for partition pruning, dramatically reducing latency and I/O.²
- **Transactional Scope:** Transactions in file-based formats are typically scoped to a single table. Atomically updating two related tables (e.g., an orders table and an inventory table) is complex and often requires an external coordination service. Because DuckLake's operations are standard SQL transactions, it naturally supports fully ACID-compliant, multi-table transactions. An UPDATE to one table and an INSERT to another can be wrapped in a single BEGIN/COMMIT block, ensuring that the changes are applied atomically across the entire database.²
- **Concurrency Control:** Iceberg and Delta Lake typically use an optimistic concurrency control mechanism based on atomic swap operations on a pointer file in storage. This can become a bottleneck under high concurrent write loads. DuckLake shifts the concurrency bottleneck to the catalog database's transaction processing engine. A modern PostgreSQL instance can handle thousands of transactions per second, enabling a much higher degree of write concurrency than file-based coordination can typically

achieve.²

The following table provides a detailed comparison of the architectural approaches of DuckLake, Apache Iceberg, and Delta Lake.

Table 1: Feature Comparison of Lakehouse Formats

Feature	DuckLake	Apache Iceberg	Delta Lake
Metadata Storage	SQL Database (e.g., PostgreSQL, MySQL, DuckDB)	Files in Object Storage (JSON, Avro)	Files in Object Storage (JSON, Parquet)
Transaction Scope	Multi-table, database-level ACID	Single-table ACID	Single-table ACID
Concurrency Model	Database Transaction Locks (e.g., MVCC)	Optimistic Concurrency (Atomic Swap on Pointer)	Optimistic Concurrency (Atomic Swap on Log File)
Small File Handling	Data Inlining (writes staged in DB) & SQL-based compaction	Requires periodic compaction jobs for data & metadata files	Requires periodic compaction jobs (e.g., OPTIMIZE)
Schema Evolution	Transactional DDL via SQL ALTER TABLE	Atomic metadata pointer updates	Atomic commits to transaction log
Query Planning	Single SQL query to catalog for file list	Multi-hop file reads (metadata → manifest list → manifests)	Read transaction log for file list
Primary Dependency	SQL Database with ACID support	Object Storage, Catalog Service (e.g., Glue, Nessie)	Object Storage, Catalog Service (optional but recommended)

Hands-on Tutorial: Building Your First DuckLake

This tutorial provides a step-by-step walkthrough for creating and interacting with your first DuckLake instance. It uses a minimal local setup, with DuckDB serving as both the query client and the metadata catalog, demonstrating the core concepts in a simple, self-contained environment.⁹

Prerequisites and Setup

Before beginning, ensure the following components are installed on your system. This guide assumes a Unix-like environment (Linux or macOS) or Windows with WSL2.

- **DuckDB CLI:** The command-line interface is the simplest way to interact with DuckLake. It can be installed with a single command.⁷ Ensure you are using a recent version that includes the DuckLake extension (v1.3.1 or newer).⁹

```
Shell
curl https://install.duckdb.org | sh
```

- **Python (Optional):** For programmatic interaction, Python 3.9 or higher is recommended.¹² The duckdb Python package can be installed via pip:

```
Shell
pip install "duckdb>=1.3.2"
```

Step-by-Step Guide: Your First Local DuckLake

This guide will walk you through creating a DuckLake for a simple retail analytics use case, storing both metadata and data on your local filesystem.

1. Launch the DuckDB CLI

Open your terminal and start the DuckDB shell:

```
Shell
./duckdb
```

2. Install and Load the DuckLake Extension

Within the DuckDB shell, run the following SQL commands. The `INSTALL` command downloads the extension if it's not already present, and `LOAD` makes its functions

available in the current session. Recent versions of DuckDB may autoload the extension on first use, but explicitly loading it is a good practice.⁹

```
SQL
INSTALL ducklake;
LOAD ducklake;
```

3. Create and Attach Your DuckLake

The ATTACH command is the primary mechanism for creating or connecting to a DuckLake. This command specifies the type of catalog backend and its location, along with the location for the Parquet data files. If the specified catalog file does not exist, DuckLake will create it automatically.

```
SQL
-- Create a DuckLake instance.
-- Metadata will be stored in a file named 'retail.ducklake'.
-- Parquet data files will be stored in a directory named 'retail_data/'.
ATTACH 'ducklake:retail.ducklake' AS retail_lake
      (DATA_PATH 'retail_data/');
```

After running this command, you will see a `retail.ducklake` file (the DuckDB database acting as the catalog) and a `retail_data` directory in your current working directory.

4. Set the Active Database Context

This is a critical step. After attaching the DuckLake, you must tell DuckDB to use it as the active database for subsequent commands. Failure to do so will result in tables being created in DuckDB's default in-memory database instead of your DuckLake.¹²

```
SQL
USE retail_lake;
```

5. Create Schemas and Tables

With the `retail_lake` context active, you can now use standard SQL DDL to define the structure of your lakehouse.


```
SQL
CREATE SCHEMA IF NOT EXISTS sales;

CREATE TABLE sales.orders (
  order_id BIGINT,
  customer_id BIGINT,
  order_date DATE,
  total_amount DECIMAL(10, 2)
);
```

6. Ingest Sample Data

Use a standard INSERT INTO statement to populate your table. Each INSERT statement will create a new snapshot in the DuckLake catalog.

```
SQL
INSERT INTO sales.orders VALUES
(101, 1, '2025-07-15', 150.75),
(102, 2, '2025-07-16', 89.99),
(103, 1, '2025-07-16', 230.00);
```

7. Query Your Data

You can now query your DuckLake table using standard SQL.

```
SQL
SELECT
  customer_id,
  COUNT(order_id) AS number_of_orders,
  SUM(total_amount) AS total_spent
FROM sales.orders
GROUP BY customer_id
ORDER BY total_spent DESC;
```

Working with Snapshots and Time Travel

Every modification to a DuckLake is captured as an atomic, immutable snapshot. This powerful feature enables auditing, debugging, and rollback capabilities.

- **Listing Snapshots:** You can view the complete history of your DuckLake using the `ducklake_snapshots()` table function. The first argument is the name you used in the ATTACH command.¹³

SQL

```
FROM ducklake_snapshots('retail_lake');  
-- Expected output will show a series of snapshots, one for each operation  
(schema creation, table creation, insert).
```

- **Time Travel Queries:** Let's perform an update and then use time travel to see the state of the table before the change.

SQL

```
-- First, get the current max snapshot_id  
SELECT max(snapshot_id) FROM ducklake_snapshots('retail_lake');  
-- Let's assume the result is 3. This is the version with the initial insert.  
  
-- Now, update a record  
UPDATE sales.orders SET total_amount = 99.99 WHERE order_id = 102;  
  
-- Query the current state (will show the updated value)  
SELECT * FROM sales.orders WHERE order_id = 102;  
  
-- Query the previous state using the snapshot ID  
SELECT * FROM sales.orders AT (VERSION => 3);
```

The second SELECT statement will return the original value of 89.99 for order 102, demonstrating the ability to query historical data instantly.¹⁵

Common First-Time User Gotchas

- **File System Permissions:** When creating a DuckLake on a local filesystem, the DuckDB process needs write permissions for both the directory where the catalog file (.ducklake) will be created and the DATA_PATH directory. An IO Error: Access is denied message during the ATTACH command typically points to a permissions issue.¹⁷
- **Forgetting the USE Command:** A frequent mistake is forgetting to execute USE <ducklake_name>; after attaching. This causes all subsequent CREATE TABLE or INSERT statements to target the wrong database, leading to confusion when the tables do not appear in the DuckLake.
- **Unsupported Constraints:** Users accustomed to traditional relational databases may try to define PRIMARY KEY or FOREIGN KEY constraints. It is important to remember that DuckLake, like other lakehouse formats, does not currently enforce these. Only NOT NULL is supported. Attempting to use other

constraints will result in a parser error.⁹

- **Path Specificity:** The DATA_PATH should be a path to a directory, not a file. The catalog path in the ATTACH string should point to the desired location of the catalog database file.

Part II: Technical Reference and Operations

This part serves as the definitive technical reference for developers and operators, covering the full SQL API, performance optimization strategies, and day-to-day operational procedures.

Complete SQL Reference and API Documentation

This section provides an exhaustive reference for all DuckLake-specific SQL commands, functions, and parameters available through the DuckDB extension. It is intended for developers and data engineers who require detailed syntactic and functional information

DuckLake-Specific SQL Extensions

The core interactions with DuckLake are managed through extensions to standard SQL syntax, primarily the ATTACH command and the AT clause for time travel.

The ATTACH Command

The ATTACH statement is the entry point for all DuckLake operations. It connects a DuckDB session to a DuckLake catalog, creating the catalog if it does not already exist.

- **General Syntax:**

```
SQL
ATTACH 'ducklake:{catalog_specifier}' AS {alias} ( [option1 'value1', option2
'value2', ...] );
```

- **Catalog Specifiers:** The {catalog_specifier} defines the backend database for the metadata catalog ⁸:
 - **DuckDB (local file):** metadata.ducklake
 - **SQLite (local file):** sqlite:metadata.sqlite
 - **PostgreSQL:** postgres:dbname=ducklake_catalog host=your_host user=...
 - **MySQL:** mysql:db=ducklake_catalog host=your_host user=...
 - **MotherDuck Catalog:** md:__ducklake_metadata_<database_name> ¹⁹
- **Key ATTACH Options** ¹⁴:
 - **DATA_PATH** '{path}': **Required on creation.** Specifies the root location for Parquet data files (e.g., a local directory 'data/' or an object storage path

's3://my-bucket/data/').

- **ENCRYPTED**: A boolean flag (true/false). When set to true, enables Parquet encryption for all data files written to the DuckLake. Keys are managed automatically within the catalog.²¹
- **METADATA_SCHEMA** '{schema_name}': Specifies the schema within the catalog database where DuckLake's internal metadata tables will be stored. Defaults to main.¹⁴
- **READ_ONLY**: A boolean flag. Opens the DuckLake in read-only mode, preventing any modifications.

Time Travel Syntax (AT Clause)

DuckLake supports querying historical data by appending an AT clause to a SELECT statement.

- **Syntax:**

```
SQL
SELECT... FROM {table_name} AT ( {version_specifier} );
```

- **Version Specifiers:**

- **VERSION** ⇒ {snapshot_id}: Queries the table as it existed at the specified integer snapshot ID.¹⁵
- **TIMESTAMP** ⇒ {timestamp_literal}: Queries the table as it existed at a specific point in time.³ DuckLake will resolve this to the latest snapshot at or before the given timestamp.

Table Functions for Introspection and Auditing

DuckLake provides a suite of table-valued functions for inspecting the catalog's metadata, which are essential for monitoring, auditing, and building advanced data pipelines. All functions take the DuckLake's attached alias as their first argument.¹³

- **ducklake_snapshots(catalog VARCHAR)**: Returns a table detailing the history of all snapshots in the catalog.
 - **Columns**: snapshot_id (BIGINT), snapshot_time (TIMESTAMP WITH TIME ZONE), schema_version (BIGINT), changes (MAP).
 - **Usage**: Essential for identifying snapshot IDs for time travel and understanding the timeline of changes.¹³
- **ducklake_table_info(catalog VARCHAR)**: Provides high-level statistics for all tables in the DuckLake.
 - **Columns**: table_name (VARCHAR), schema_id (BIGINT), table_id (BIGINT),

file_count (BIGINT), file_size_bytes (BIGINT), etc.

- **Usage:** Useful for monitoring table growth, identifying tables with a large number of files (potential candidates for compaction), and getting a quick overview of the lakehouse structure.¹³
- **ducklake_table_changes(catalog VARCHAR, schema_name VARCHAR, table_name VARCHAR, start_snapshot, end_snapshot):** Returns a detailed, row-level change data capture (CDC) stream between two snapshot versions or timestamps.
 - **Columns:** snapshot_id (BIGINT), rowid (BIGINT), change_type (VARCHAR, e.g., 'insert', 'delete'), plus all columns of the target table.
 - **Usage:** The core function for building incremental data pipelines, auditing changes, and understanding data lineage.¹³
- **ducklake_table_insertions(...)** and **ducklake_table_deletions(...):** These are specialized, more performant versions of ducklake_table_changes that return only inserted or deleted rows, respectively.¹³
- **ducklake_list_files(catalog VARCHAR,...):** Returns a list of the specific data and delete files that constitute a table at a given snapshot. This is useful for integrations with external query engines that may need to read the Parquet files directly.²⁰

Maintenance and Operational Commands

These commands are used for the ongoing maintenance and optimization of a DuckLake instance. They are typically executed as part of scheduled operational jobs.

- **ducklake_cleanup_old_files(catalog VARCHAR,...):** Performs garbage collection by deleting Parquet files from object storage that are no longer referenced by any retained snapshot.
 - **Parameters:** cleanup_all (BOOLEAN), dry_run (BOOLEAN), older_than (TIMESTAMP).
 - **Usage:** This is a crucial step for managing storage costs. It should be run after ducklake_expire_snapshots.¹³
- **ducklake_expire_snapshots(catalog VARCHAR,...):** Removes snapshot metadata from the catalog database. This makes historical data before the expired snapshots inaccessible and marks the associated data files as candidates for garbage collection.
 - **Parameters:** versions (UBIGINT), older_than (TIMESTAMP).
 - **Usage:** Manages the size of the metadata history and defines the data retention policy for time travel.¹³
- **ducklake_merge_adjacent_files(catalog VARCHAR):** Performs data

compaction by merging smaller Parquet files into larger, more optimal ones.

- **Usage:** Improves read performance for tables that receive frequent small writes. This directly addresses the "small file problem" at the data layer.¹³

API and Language Bindings

DuckLake can be managed programmatically through any of DuckDB's client APIs. The interaction model is consistent: connect to a database and execute the DuckLake SQL commands as text strings.

- **Python Example:**

```
Python
import duckdb

# Connect to an in-memory DuckDB instance to act as the client
con = duckdb.connect()

# Install and load the extension
con.execute("INSTALL ducklake;")
con.execute("LOAD ducklake;")

# Attach a DuckLake using PostgreSQL as the catalog
# Assumes a secret named 'pg_creds' has been created
attach_sql = """
ATTACH 'ducklake:postgres:dbname=ducklake_catalog' AS my_lake (
    DATA_PATH 's3://my-bucket/data/'
);
"""
con.execute(attach_sql)

# Set context and query the DuckLake
con.execute("USE my_lake;")
results = con.execute("SELECT * FROM sales.orders LIMIT 10;").fetchdf()
print(results)
```

The API interaction model is straightforward, leveraging the existing, mature client libraries for DuckDB, which are available for a wide range of languages including C++, Java, Node.js, and more.²²

Performance Tuning & Best Practices

Optimizing DuckLake performance involves a shift in mindset from tuning distributed file systems to applying principles of traditional database performance management, combined with an understanding of the lakehouse architecture. The key is to leverage the strengths of the SQL catalog for metadata operations while ensuring the physical data layout is efficient for analytical queries.

Query Optimization Techniques

DuckLake's primary performance advantage stems from its ability to make metadata-intensive operations significantly faster than its file-based counterparts.

- **Leveraging the SQL Catalog for Pruning:** The most critical optimization is DuckLake's use of the SQL catalog for query planning. When a query with a WHERE clause is submitted, the DuckDB query optimizer translates filters on partition columns into a SQL query against the catalog database's metadata tables. The catalog database can then use its native indexes to very quickly identify the exact subset of data files that need to be scanned, avoiding a costly full scan of all table metadata files.³ This index-based partition discovery is fundamentally more efficient than sequentially scanning metadata files in object storage.
- **Data Skipping with Column Statistics:** DuckLake automatically collects and stores column-level statistics (such as min/max values and null counts) for each Parquet file within the catalog database.¹⁵ During query planning, the optimizer uses these statistics to perform data skipping (also known as predicate pushdown or vertical partitioning). For example, if a query contains the filter WHERE product_id = 123, the optimizer will query the catalog to find which files have a min_value less than or equal to 123 and a max_value greater than or equal to 123 for the product_id column. Any files that do not meet this criterion can be skipped entirely, dramatically reducing the amount of data read from object storage.

Partitioning Strategies and Guidelines

Effective partitioning is crucial for query performance in any large-scale analytical system. DuckLake supports hidden partitioning, which optimizes the physical data layout without complicating the user's query experience.

- **Hidden Partitioning:** DuckLake allows tables to be partitioned by one or more columns (e.g., PARTITIONED BY (event_date, country)). This organizes the Parquet files in the data store into a directory structure based on the partition key values. This layout is "hidden" from the user, meaning they do not need to include the partition columns in their WHERE clauses for pruning to occur. The query planner automatically leverages the partitioning scheme based on filters applied to those columns.¹

- **Best Practices for Partitioning:**

- **Choose Low-Cardinality Columns:** Select partition keys that have low to moderate cardinality. Columns like date, month, region, or customer_tier are good candidates. Partitioning on a high-cardinality column (e.g., user_id in a table with millions of users) will create an excessive number of partitions and small files, harming performance.
- **Align with Query Patterns:** Analyze the most common and performance-critical query patterns. Partition keys should align with the columns most frequently used in WHERE clause filters.
- **Avoid Over-Partitioning:** Creating too many partitions can be as detrimental as having none. Each partition adds some metadata overhead. A good rule of thumb is to ensure each partition contains a substantial amount of data, ideally hundreds of megabytes to gigabytes. For time-series data, partitioning by day or month is often more effective than partitioning by hour or minute.

File Size and Compaction

Managing the size and number of data files is a critical aspect of maintaining a healthy and performant data lake.

- **The "Small File Problem":** Analytical query engines like DuckDB perform best when reading large, contiguous blocks of columnar data. A table composed of thousands of tiny Parquet files will suffer from poor read performance due to the overhead of opening each file, reading its metadata footer, and issuing separate I/O requests. This is a well-known problem in all data lake systems.¹¹
- **Data Inlining for Writes:** DuckLake offers an innovative feature called "data inlining" to mitigate the small file problem for write-heavy workloads. When enabled, very small writes (e.g., single-row inserts from a streaming source) are not immediately written to a new Parquet file. Instead, the data is stored directly ("inlined") within the SQL catalog database itself. These inlined rows are immediately available for querying. Periodically, a background process consolidates these small batches of inlined data and flushes them to larger, more optimal Parquet files in the data store.² This mechanism transforms high-frequency, low-volume updates from a performance bottleneck into an efficient operation, a key differentiator from Iceberg and Delta Lake.
- **Manual Compaction for Reads:** For read-heavy tables that may have accumulated small files over time (e.g., from historical batch loads), operators should periodically run the `ducklake_merge_adjacent_files(catalog)` command. This command will scan a table for small files and rewrite them into a smaller

number of larger files, improving subsequent query performance.¹⁵ A common best practice is to target file sizes in the range of 256 MB to 1 GB.

Concurrency and Anti-Patterns

DuckLake's architecture enables a "multiplayer DuckDB" experience, allowing multiple clients to read from and write to the same data lakehouse concurrently.⁸

- **Concurrency Model:** DuckLake uses a model best described as "optimistic concurrency control with intelligent merge resolution".⁶ When two transactions attempt to commit concurrently, one will fail at the database level due to a primary key violation on the snapshot ID. Instead of simply failing the user's transaction, DuckLake's client library catches this specific error. It then analyzes the logical changes of the conflicting transactions. If the changes are non-conflicting (e.g., two separate INSERT operations into the same table, or updates to different tables), the client will automatically retry the transaction against the new state of the catalog. This intelligent resolution allows many concurrent analytical operations to succeed without manual intervention.
- **Anti-Patterns to Avoid:**
 - **Using DuckLake for OLTP:** Despite its support for ACID transactions, DuckLake is an OLAP (Online Analytical Processing) system optimized for scans, aggregations, and bulk operations. It is not a replacement for an OLTP (Online Transaction Processing) database like PostgreSQL. Workloads characterized by high-frequency, single-row UPDATES and DELETES with strict latency requirements are not a good fit.²²
 - **Neglecting Catalog Performance:** The SQL catalog database is the performance linchpin of the entire system. Using an under-provisioned or poorly configured database for the catalog will create a bottleneck that limits the scalability of the entire lakehouse. For production workloads, the catalog should be hosted on a robust, production-grade database instance with sufficient resources and proper monitoring.²
 - **Creating Unnecessary Snapshots:** While creating snapshots is cheap, every transaction creates one. Avoid wrapping single-row inserts in individual transactions within a loop. It is far more efficient to batch writes into larger transactions, which saturates network throughput and creates fewer, more meaningful snapshots.²²

Operations & Monitoring in Production

Maintaining a production-grade DuckLake deployment requires a robust operational strategy focused on the health of its two core components: the SQL

catalog and the object data store. This section provides a guide to "day 2" operations, including health checks, backup and recovery, routine maintenance, and troubleshooting.

Health Checks and Monitoring Queries

Continuous monitoring is essential to ensure the reliability and performance of your DuckLake. Since the catalog database is a potential single point of failure, its health is paramount.²

- **Catalog Database Health:** The catalog should be monitored using standard database administration tools. For a PostgreSQL backend, this includes tracking key metrics such as:
 - CPU and Memory Utilization
 - Disk I/O and available storage space
 - Active connections and transaction throughput
 - Query latency, particularly for metadata-related queries
 - Lock contention, using views like `pg_stat_activity` and `pg_locks` to identify long-running or blocked transactions.
- **DuckLake-Specific Monitoring Queries:** A set of SQL queries should be executed periodically against the DuckLake catalog to monitor its internal state. These can be integrated into standard monitoring and alerting systems.
 - **Monitor Snapshot Growth:** A rapid, unexpected increase in snapshots could indicate a runaway process.

SQL

```
-- Get the total number of snapshots and the timestamp of the latest one
SELECT
    COUNT(snapshot_id) AS total_snapshots,
    MAX(snapshot_time) AS latest_snapshot_time
FROM ducklake_snapshots('your_lake_alias');
```

- **Track Table Size and File Counts:** This helps identify tables that are growing quickly or are suffering from the small file problem.

SQL

```
-- Get file count and total size for each table
SELECT
    table_name,
    file_count,
    file_size_bytes
FROM ducklake_table_info('your_lake_alias')
ORDER BY file_size_bytes DESC;
```

- **Check for Stale Statistics:** Ensure column statistics are being properly maintained by querying the statistics tables directly. Missing statistics can lead to suboptimal query plans.¹⁵

SQL

```
-- This query is conceptual and depends on direct access to the catalog schema
-- Check for tables that have files but no corresponding column stats
SELECT table_name FROM my_catalog.ducklake_table t
WHERE t.table_id IN (SELECT table_id FROM my_catalog.ducklake_data_file)
AND t.table_id NOT IN (SELECT table_id FROM
my_catalog.ducklake_file_column_statistics);
```

Backup and Recovery Procedures

A comprehensive backup and recovery strategy for DuckLake must address both the catalog database and the data files in object storage.

- **Two-Component Backup Strategy:**

1. **Catalog Database Backup:** The SQL catalog contains the entire history and structure of the lakehouse and must be backed up regularly. Use the native backup tools provided by your chosen database system (e.g., `pg_dump` for PostgreSQL or point-in-time recovery features in managed cloud services like AWS RDS). This is a well-established practice for any production database.³
2. **Data Store Backup:** The Parquet files in the object store must also be protected. Best practice is to enable versioning on your object storage bucket (e.g., S3 Versioning). This protects against accidental deletion or corruption of data files. For disaster recovery, configure cross-region replication to maintain a copy of your data in a separate geographical location.

- **Recovery Process:**

1. **Restore Data Store:** If data files are lost, restore the object storage bucket from your backup or to a specific point in time using the provider's versioning capabilities.
2. **Restore Catalog Database:** Restore the SQL catalog database from its backup to a point in time that is consistent with the state of the restored data store.
3. **Validation:** After restoration, connect to the DuckLake and run validation queries. Use time travel (`AT TIMESTAMP...`) to query the state at the time

of the backup and verify that key tables have the expected row counts and data integrity.

Catalog Maintenance

Regular maintenance is essential for controlling costs and ensuring sustained performance. These tasks should be automated and scheduled.

Table 5: DuckLake Maintenance Commands

Command	Description	Key Parameters	When to Use
ducklake_expire_snapshots	Removes old snapshot metadata from the catalog database, enforcing the data retention policy.	older_than, versions	Scheduled regularly (e.g., weekly/monthly) to prune history and enable garbage collection.
ducklake_cleanup_old_files	Physically deletes orphaned Parquet files from object storage that are no longer referenced by any retained snapshot.	dry_run, older_than	Run immediately after ducklake_expire_snapshots to reclaim storage space and reduce costs.
ducklake_merge_adjacent_files	Compacts small data files into larger, more optimal files for reading.	catalog	Scheduled periodically (e.g., daily/weekly) on tables with frequent, small writes to improve query performance.

A typical maintenance workflow would be a scheduled job (e.g., a weekly Airflow DAG) that first expires snapshots older than the defined retention period, and then immediately runs the file cleanup command. A separate, more frequent job might run the file compaction command on specific high-volume tables.

Troubleshooting Common Issues

- **Catalog Connection Failures:**
 - **Diagnosis:** Check network connectivity between the client and the database server (firewalls, security groups, VPC routing). Verify

credentials (connection string, username/password, secrets). Ensure the catalog database is running and healthy.

- **Concurrent Write Failures / Deadlocks:**

- **Diagnosis:** High rates of transaction failures or deadlocks in the catalog database logs (e.g., "deadlock detected" in PostgreSQL) indicate severe write contention.²⁹ While DuckLake's intelligent retry mechanism handles many cases⁶, extreme contention may overwhelm it.
- **Workaround:** For massive parallel ingestion jobs (e.g., ingesting thousands of tables at once), consider introducing an external locking mechanism or batching operations to reduce the number of concurrent transactions hitting the catalog.³⁰ Check the DuckLake GitHub Issues for known concurrency bugs.³¹

- **Query Performance Degradation:**

- **Diagnosis Checklist:**
 1. **Check Catalog Health:** Is the catalog database under-resourced or experiencing high load?
 2. **Analyze Table Info:** Use `ducklake_table_info` to check for an explosion in file counts for the queried tables.
 3. **Review Maintenance Jobs:** Have the snapshot expiration, cleanup, and compaction jobs been running successfully? A backlog of small files is a common cause of slow reads.
 4. **EXPLAIN the Query:** Use `EXPLAIN` or `EXPLAIN ANALYZE` on the slow query to inspect the query plan. Verify that partition pruning and data skipping are being applied as expected.³²

Part III: Integration, Migration, and Security

This part focuses on placing DuckLake within the broader data ecosystem, providing practical guidance on connecting it to other tools, migrating existing data, and securing the entire platform. This demonstrates DuckLake's design philosophy as a composable component that integrates cleanly into the modern data stack, rather than attempting to be a monolithic, all-in-one solution.

The Integration Cookbook: DuckLake in the Modern Data Stack

DuckLake's use of standard SQL and JDBC/ODBC interfaces allows it to integrate with a wide range of common data tools. This section provides practical recipes for connecting DuckLake to popular BI, ETL, and orchestration platforms.

ETL/ELT Pipeline Patterns

DuckLake's architecture, which separates storage and compute and provides a powerful in-process SQL engine, is exceptionally well-suited to the modern ELT (Extract, Load, Transform) pattern. This approach prioritizes loading raw data into the lakehouse quickly and performing transformations in-place using the power of the target system.³⁷

- **Real-Time Streaming Ingestion (ELT):**

- **Architecture:** A streaming source (like Apache Kafka or AWS Kinesis) sends events to a lightweight consumer application. The consumer batches these events and writes them to a DuckLake table in near real-time.
- **Implementation:** Tools like Estuary Flow provide connectors that can stream data from sources like PostgreSQL CDC, Kafka, or webhooks and materialize them directly into a MotherDuck-managed DuckLake. This process leverages DuckLake's data inlining feature to efficiently handle high-frequency, small-batch writes with exactly-once semantics.¹⁹
- **Dataflow:** Source → Kafka → Streaming Consumer (DuckDB client) → INSERT → DuckLake (Catalog DB + Object Storage).

- **Scalable Batch Ingestion (ELT):**

- **Architecture:** A periodic batch job extracts a large volume of data from a source system (e.g., an operational database or SaaS API). The raw data is first landed in object storage (e.g., as Parquet or CSV files). A second step then loads this data into a raw table in DuckLake.
- **Implementation:** An orchestration tool like Airflow or Dagster triggers the job. The data can be loaded into DuckLake efficiently using the COPY statement or INSERT INTO... SELECT FROM read_parquet('s3://...'). Subsequent transformations are then run as SQL queries inside DuckLake.¹⁵
- **Dataflow:** Source DB → Batch Export Job → Raw Files in S3 → COPY command → DuckLake Raw Table → SQL Transformations → DuckLake Curated Tables.

Orchestration and Transformation

- **dbt (data build tool):**

- **Configuration:** Use the dbt-duckdb adapter. In your profiles.yml, configure the path to point to your DuckLake. For MotherDuck, this is a connection string like md:my_database. For a self-hosted DuckLake, this would be the path to the local DuckDB file that can attach the lake.⁴¹

- **Workflow:** Define your raw data in DuckLake as sources in dbt. Then, write standard dbt SQL models to transform this data into staging and marts layers. dbt run will execute these transformations directly within the DuckDB engine against your DuckLake data.
- **Airflow:**
 - **Configuration:** Install the airflow-provider-duckdb. In the Airflow UI, create a new connection of type "DuckDB". For MotherDuck, provide your service token. For a self-hosted DuckLake, provide the file path to the client database.⁴²
 - **DAG Implementation:** Use the DuckDBHook inside a @task decorated function to get a connection and execute SQL commands against your DuckLake, such as running ingestion jobs or maintenance tasks.⁴²
- **Dagster:**
 - **Configuration:** Use the dagster-duckdb library. In your Dagster definitions, configure a DuckDBResource and set the database parameter to the path of your DuckLake's client database file.⁴⁴
 - **Asset Implementation:** Within your Dagster assets, accept the DuckDBResource as an argument. Use duckdb.get_connection() to execute queries for ingesting data, performing transformations, and creating downstream tables in DuckLake.⁴⁴

The Migration Playbook: Moving to DuckLake

Migrating an existing data platform to DuckLake requires a structured approach that considers the source format, data validation, and rollback strategies. The key advantage when migrating from other open table formats is the potential for a metadata-only migration, avoiding a costly and time-consuming physical data rewrite.

Migration from Iceberg, Delta Lake, or Hudi

Since DuckLake, Iceberg, Delta, and Hudi all use Apache Parquet as their underlying data file format, migration can be significantly streamlined. The primary task is to translate the metadata from the source format's structure into the DuckLake SQL catalog schema.

- **Metadata-Only Migration Path:** The DuckLake team has stated that metadata-only import/export from Iceberg is on the roadmap.⁵ This approach would involve a tool that reads the source table's metadata (e.g., Iceberg's manifest files) and writes the equivalent records into the DuckLake catalog tables (ducklake_data_file, ducklake_column, etc.), pointing to the *exact same Parquet files* in object storage. This avoids any data movement.

- **Conceptual Migration Steps:**
 1. **Freeze Source Writes:** Halt all ingestion pipelines writing to the source table to ensure a consistent state for migration.
 2. **Translate Metadata:** Use a migration utility (or a custom script) to perform the metadata translation. This script would parse the source table's transaction log or manifest list to get the current schema, partition spec, and list of active data files. It would then generate and execute the necessary INSERT statements into the DuckLake catalog tables to represent this state.
 3. **Validate Data:** Perform rigorous validation to ensure the migration was successful (see section 7.2).
 4. **Cutover:** Re-point all downstream consumers (BI tools, data applications) from the source table to the new DuckLake table.
 5. **Resume Writes:** Restart ingestion pipelines, now targeting the DuckLake table.
- **Manual Migration using ducklake_add_data_files:** In the absence of an automated tool, a manual migration is possible. The `ducklake_add_data_files` function, introduced in DuckLake 0.2, allows you to register existing Parquet files into a DuckLake table. This is particularly useful for files that were not written by DuckLake and lack its internal field IDs. The process involves using "name mapping" to map the column names in the Parquet files to the columns in the DuckLake table.²⁰

SQL

```
-- 1. Create the target table in DuckLake
CREATE TABLE my_lake.migrated_table (id BIGINT, name VARCHAR);

-- 2. Register the existing Parquet files from the old table
-- The function will map columns by name.
CALL ducklake_add_data_files('my_lake', 'migrated_table',
's3://old-iceberg-table/data/*.parquet');
```

Data Validation Strategies

Thorough validation is non-negotiable for any migration.

- **Aggregate Queries:** The most basic validation is to compare aggregate metrics between the source and target tables. These queries should be run on both systems and the results must match exactly.

SQL

```
SELECT
  COUNT(*) AS row_count,
  COUNT(DISTINCT primary_key_column) AS distinct_keys,
  SUM(numeric_column) AS total_sum,
  AVG(numeric_column) AS average_value
FROM your_table;
```

- **Row-Level Checksums:** For a more granular check, generate a checksum or hash of the concatenated values for each row. Comparing the set of checksums between the source and target can identify subtle data discrepancies or missing/extra rows.
- **Incremental Validation:** After the initial load, use DuckLake's `ducklake_table_changes` function to compare incremental updates. A parallel pipeline can run, feeding a CDC stream from the source and the output of `table_changes` into a validation tool to ensure they match.

Rollback Procedures

A clear rollback plan is essential to de-risk the migration cutover.

- **Dual-Write Strategy:** For a limited period before and after the cutover, configure ingestion pipelines to write to *both* the old and new systems. This maintains the old system as a live, up-to-date backup, allowing for an immediate rollback if a critical issue is discovered in the new DuckLake system.
- **Snapshot-Based Recovery:** DuckLake's snapshotting provides a powerful and fast rollback mechanism. Before the cutover, take note of the last stable snapshot ID. If a post-migration data corruption event occurs (e.g., due to a faulty ingestion job), you can instantly revert the table to its pre-cutover state for read queries using time travel. While a full database-level regression is not yet a built-in command, this allows for immediate mitigation for analytical consumers.¹⁶

Table 6: Migration Decision Matrix

Source System	Key Considerations	Recommended Strategy	Effort Level
Apache Iceberg / Delta Lake	Data files are compatible (Parquet). Metadata structure is different.	Metadata-Only Migration: Use future tooling or manual <code>ducklake_add_data_files</code> . Avoids data movement.	Low to Medium
Traditional RDBMS (e.g., PostgreSQL, MySQL)	Data and schema must be extracted. No compatible file format.	Full ETL/ELT: Extract data to Parquet files in object storage, then COPY into DuckLake.	High
CSV/Parquet Files in S3	Data files are compatible, but no transactional history or schema.	Direct Ingestion: Create a new DuckLake table and use <code>CREATE TABLE AS SELECT FROM read_parquet(...)</code> or COPY to ingest.	Low

Security Implementation Guide

Securing a DuckLake environment involves a multi-layered approach that leverages the native security features of its components: the SQL catalog database and the cloud object store. The core architectural principle is to centralize access control at the metadata layer, creating a simplified yet powerful security model.

Authentication and Authorization

The primary security model for DuckLake is elegantly simple: it delegates authentication and authorization to the underlying catalog database.¹¹ Access to the data lake is synonymous with access to its metadata.

- **Catalog-Based Access Control:** If you use PostgreSQL as your catalog, you can use standard PostgreSQL roles, users, and GRANT/REVOKE statements to

control who can connect to the catalog and what operations they can perform. By granting read-only access to a specific database user, you effectively grant read-only access to the entire DuckLake for any client connecting as that user. This model is familiar to database administrators and allows for fine-grained control.

- **MotherDuck Integrated Authentication:** When using a managed DuckLake on MotherDuck, the platform handles authentication and authorization. Users and service accounts are managed within a MotherDuck organization. MotherDuck acts as a credential broker; a user who is granted access to a DuckLake database does not need direct credentials to the underlying S3 bucket. MotherDuck provides temporary, scoped credentials (e.g., presigned URLs) for storage access on behalf of the authenticated user, simplifying credential management and enhancing security.⁵
- **Credential Management with CREATE SECRET:** For self-hosted deployments, credentials for accessing object storage (e.g., AWS keys) or the remote catalog database must be provided to the DuckDB client. The recommended approach is to use DuckDB's secret manager. The CREATE OR REPLACE SECRET command allows you to securely store these credentials, which are then referenced by name in connection strings. When used with MotherDuck, secrets are fully encrypted and stored in the cloud.⁷

SQL

```
-- Create a secret for S3 access using the local AWS credential chain
CREATE OR REPLACE PERSISTENT SECRET s3_creds (TYPE S3, PROVIDER
credential_chain);

-- Create a secret for a PostgreSQL catalog
CREATE SECRET pg_creds (TYPE postgres, HOST '...', PASSWORD '...');
```

Network Security Configurations

Proper network configuration is essential to protect a DuckLake deployment from external threats.

- **Securing the Catalog Database:** For production environments, the catalog database should never be exposed to the public internet. It should be deployed within a private network (e.g., an AWS VPC) and protected by firewalls or security groups that restrict inbound traffic to only trusted sources, such as application servers or specific IP ranges. All connections to the catalog should be encrypted using TLS/SSL.
- **Securing Object Storage:** To ensure that data traffic between your compute

resources and object storage remains within the cloud provider's network, use private network endpoints (e.g., AWS S3 Gateway Endpoints or Azure Private Link for Storage). This prevents data from traversing the public internet, reducing exposure and potentially lowering data transfer costs.

- **MotherDuck Private Link:** For enterprise customers requiring the highest level of network security, MotherDuck offers connectivity via AWS PrivateLink. This creates a secure, private connection between your VPC and the MotherDuck service, ensuring that all traffic remains on the AWS backbone network.⁴⁷

Encryption and Key Management

DuckLake's built-in encryption capability is one of its most powerful security features, enabling a true zero-trust data architecture.

- **Zero-Trust Architecture:** When encryption is enabled, every Parquet file written by DuckLake is encrypted with a unique, automatically generated encryption key. Crucially, these data encryption keys are stored exclusively within the secured metadata catalog database.⁶ This separation of encrypted data (in object storage) from encryption keys (in the database) means that the data files themselves can be stored on completely untrusted infrastructure. Even if an attacker gains access to the object storage bucket, the Parquet files are useless without also compromising the catalog database to retrieve the keys.¹⁶
- **Enabling Encryption:** Encryption is enabled with a simple flag during the initial ATTACH command that creates the DuckLake:

SQL

```
ATTACH 'ducklake:encrypted.ducklake' (DATA_PATH 'untrusted_s3_bucket/',  
ENCRYPTED);
```

Once enabled, all subsequent read and write operations are transparently encrypted and decrypted by the DuckDB client, which fetches the necessary keys from the catalog on-the-fly.²¹

- **Key Management:** The key management process is fully automated. New keys are generated for each write operation, ensuring that each file is encrypted with its own key. The user does not need to manage the lifecycle of these keys; this is handled internally by DuckLake.

Compliance and Audit Trail Configuration

DuckLake's features provide a strong foundation for meeting compliance requirements and establishing robust audit trails.

- **Supporting GDPR and HIPAA:** While compliance is a broad topic involving policies and procedures, DuckLake's technical features can help organizations meet the data protection requirements of regulations like GDPR and HIPAA.⁴⁹ The strong, zero-trust encryption model helps secure Protected Health Information (PHI) and Personally Identifiable Information (PII) at rest. The catalog-based authorization model provides the fine-grained access control necessary to enforce the principle of least privilege. MotherDuck itself has achieved GDPR compliance, providing a compliant platform for managed DuckLake deployments.⁵¹
- **Building a Complete Audit Trail:** Because every change in DuckLake is an atomic transaction that creates a new snapshot, the catalog itself serves as a comprehensive audit log. By querying the metadata tables, an auditor can reconstruct the exact state of the data and schema at any point in time.
 - **The `ducklake_table_changes` function is the primary tool for auditing data modifications.** It provides a complete, row-level history of all inserts and deletes between any two points in time.¹⁵
 - Queries against the `ducklake_schema`, `ducklake_table`, and `ducklake_column` tables, filtered by `begin_snapshot` and `end_snapshot`, can provide a full audit trail of all schema evolution activities.²⁸

Part IV: Ecosystem, Context, and Community

This final part provides the broader context needed for strategic decision-making, including cost, limitations, real-world use cases, and how to engage with the developer community.

Cost Analysis & Optimization

Understanding the cost structure of a DuckLake deployment is critical for planning and resource management. The costs depend on the deployment model: a fully managed service like MotherDuck or a self-hosted configuration. MotherDuck offers true serverless billing for data lakes and warehouses, and provides the infrastructure on your behalf.

Cost Comparison with Alternatives

While direct TPC-DS benchmark results for DuckLake are not yet available, a conceptual cost comparison can be made based on architectural differences with

platforms like Snowflake, BigQuery, and Databricks.

- **Architecture and Pricing Model:**
 - **Snowflake & Databricks:** Both use a credit-based system where compute is billed based on virtual warehouse or cluster uptime. This model is powerful but can lead to high costs if resources are not managed carefully.⁵²
 - **BigQuery:** Offers on-demand pricing (cost per byte scanned) and capacity-based pricing (reserving slots). The serverless model is simple but can become expensive for queries that scan large amounts of unoptimized data.⁵²
 - **MotherDuck/DuckLake:** The model is a hybrid. It offers serverless (Pulse) and provisioned (Standard, etc.) compute, but its key differentiator is the ability to leverage local compute. For many analytical tasks performed by data scientists and analysts, queries can be partially or fully executed on their laptops, incurring zero cloud compute cost.²⁵ This aligns with the "small data" philosophy, suggesting that many workloads do not require a massive, expensive cloud cluster, making MotherDuck a highly cost-effective alternative for a significant portion of analytical use cases.²⁷
- **Self-Hosted vs. Managed:** A self-hosted DuckLake requires paying for the underlying infrastructure: a managed SQL database (e.g., AWS RDS for PostgreSQL), object storage (e.g., S3), and the compute nodes running the DuckDB clients. This offers maximum control but also incurs operational overhead for management, backup, and security. MotherDuck abstracts these costs into a single, managed service, trading some control for simplicity and potentially lower total cost of ownership (TCO).

Storage and Compute Optimization Strategies

- **Storage Cost Optimization:**
 - **Regular Maintenance:** Implement a regular schedule for running `ducklake_expire_snapshots` followed by `ducklake_cleanup_old_files`. This is the most critical practice for preventing storage costs from growing indefinitely due to orphaned data files.
- **Compute Cost Optimization:**
 - **Right-Size Instances:** On MotherDuck, choose the smallest compute instance that meets your workload's performance requirements. Use the serverless Pulse instance for ad-hoc and unpredictable workloads to avoid paying for idle dedicated resources.⁴⁷
 - **Leverage Partitioning:** A well-designed partitioning strategy is a direct cost-saving measure. By allowing the query engine to prune partitions and

skip files, you reduce the amount of data scanned, which lowers I/O and compute time.³

- **Embrace Dual Execution:** Encourage analysts and data scientists to use local DuckDB clients connected to MotherDuck. The distributed query planner will offload work to their local machines whenever possible, reducing cloud compute consumption.²⁵
- **Batch Writes:** When ingesting data, batch records into larger transactions rather than committing row-by-row. This is more efficient from a compute and network perspective and reduces metadata overhead.²²

Limitations, Trade-offs, and Future Direction

A transparent understanding of DuckLake's current limitations and design trade-offs is essential for making informed adoption decisions. As an emerging technology, it offers significant advantages but also comes with feature gaps and a different set of operational considerations compared to more mature formats.

When (and When Not) to Use DuckLake

- **Ideal Use Cases:**

- **Modern Data Warehousing:** DuckLake serves as an excellent foundation for a scalable, SQL-based data warehouse, particularly for organizations that find traditional cloud data warehouses to be overly complex or expensive for their needs.²²
- **Interactive, Multi-User Analytics:** The "multiplayer DuckDB" capability makes it ideal for teams of analysts and data scientists who need to concurrently query and manipulate a shared dataset.¹¹
- **Embedded Analytics and Data Apps:** MotherDuck's WASM capabilities, combined with DuckLake's performance, enable the development of highly interactive data applications where analytics run directly in the user's browser.²⁴
- **Workloads aligned with the "Small Data" Philosophy:** For the vast majority of analytics workloads that are less than 1 TB in size, DuckLake provides a simpler, faster, and more efficient solution than distributed "big data" platforms.²⁵

- **Not Suitable For:**

- **High-Throughput OLTP:** DuckLake is an OLAP system. It is not designed for the high-frequency, low-latency, single-row updates and deletes characteristic of transactional application backends. It lacks critical OLTP features like indexes and foreign key enforcement.¹¹

Performance Boundaries and Feature Gaps

- **Catalog as a Bottleneck:** The primary performance boundary of a DuckLake system is the throughput of its metadata catalog database. An under-provisioned or poorly managed catalog will limit the performance of the entire lakehouse.²
- **Feature Gaps vs. Traditional RDBMS:**
 - **No Constraints:** DuckLake does not currently support or enforce PRIMARY KEY, FOREIGN KEY, or UNIQUE constraints. Data integrity must be managed at the application or transformation layer.¹¹
 - **No Secondary Indexes:** There is no support for secondary indexes on data columns. Performance relies on partitioning, column statistics, and the raw scanning speed of the DuckDB engine.
- **Ecosystem Maturity:** Compared to Iceberg and Delta Lake, which have been in production for years and have broad support across tools like Spark, Flink, and Trino, DuckLake's ecosystem is still nascent. The primary implementation is the DuckDB extension, and wider support from other query engines is part of the future roadmap.²

Known Issues, Workarounds, and Roadmap

- **Production Readiness:** While extensively tested, it is not yet recommended for all production use cases, with general production readiness expected to mature over the course of 2025.¹¹ Early adopters should be prepared for potential bugs and breaking changes between versions.
- **Known Issues:** The official duckdb/ducklake GitHub repository's "Issues" tab is the canonical source for tracking known bugs, performance problems, and feature requests.³¹ Commonly reported issues include challenges with high-concurrency writes leading to deadlocks in the PostgreSQL catalog, and difficulties with the automatic migration process between extension versions.²⁹
- **Future Roadmap:** The development vision for DuckLake is ambitious and includes:
 - **Interoperability:** Tooling to import from and export to Apache Iceberg and Delta Lake metadata formats, enabling seamless migration and coexistence.⁵
 - **Broader Engine Support:** First-class support for other query engines like Apache Spark to read from and write to DuckLake, moving beyond the DuckDB ecosystem.¹⁹
 - **Enhanced Features:** Continued improvements to performance, concurrency, and the addition of features like row-level and column-level

security.¹⁹

Developer & Community Resources

The DuckDB and MotherDuck ecosystem is vibrant and growing, offering a wealth of resources for users and developers to get support, contribute to the projects, and extend their functionality.

Contributing and Extension Development

Both DuckDB and DuckLake are open-source projects that welcome community contributions.

- **Contributing Guide:** The primary channels for contributing to DuckLake are through its GitHub repository.
 - **Bug Reports & Issues:** Problems or bugs should be filed as issues in the duckdb/ducklake issue tracker.¹¹
 - **Feature Requests & Ideas:** Suggestions and feature requests should be started in the duckdb/ducklake discussion forum to allow for community feedback.¹¹
 - **Documentation:** Contributions to the documentation can be made via pull requests to the duckdb/duckdb-web repository. Contributors should follow the style guide outlined in the CONTRIBUTING.md file.⁶⁰
- **Extension Development:** DuckDB is highly extensible, and developers can create their own extensions to add new functionality. The duckdb/extension-template repository on GitHub provides a complete template and documentation for getting started. The process involves cloning the template, using a bootstrap script to rename the extension, adding custom C++ code, and building it against the DuckDB source tree.⁶¹
- **Custom Functions (UDFs):** A simpler way to extend DuckDB's functionality is by creating User-Defined Functions (UDFs) in a host language like Python. The duckdb.create_function() API allows a Python function to be registered and called directly from SQL. This is a powerful way to integrate custom business logic or third-party libraries into data analysis workflows without needing to build a full C++ extension.⁶²

Documentation and Training

- **Official Documentation:** The canonical sources for documentation are the DuckDB website (duckdb.org), the DuckLake website (ducklake.select), and the MotherDuck documentation (motherduck.com/docs).
- **Documentation Feedback:** The documentation is treated as code. Users who find errors or areas for improvement are encouraged to submit pull requests

or file issues in the [duckdb/duckdb-web GitHub repository](#).⁶⁰

- **Training and Certification:** While there are no formal training or certification programs at this time, a vast amount of high-quality training material is available through community and official channels. This includes official blog posts with tutorials, a rich library of videos and conference talks, and numerous third-party blog posts and guides that provide practical, real-world examples.⁹

Conclusion

DuckLake emerges not merely as another competitor in the "table format wars," but as a paradigm shift that challenges the foundational assumptions of the modern data lakehouse. By rejecting the file-based metadata model of its predecessors and returning to the proven, transactional integrity of a standard SQL database, DuckLake offers a solution that is architecturally simpler, operationally more familiar, and, for a wide range of analytical workloads, significantly more performant.

Its core strengths lie in this architectural elegance. The ability to perform true multi-table ACID transactions, the dramatic performance improvements for high-frequency writes via data inlining, and the simplified security model based on database-native authentication and zero-trust encryption are direct consequences of this design choice. It effectively outsources the most complex aspects of data management—concurrency, transactions, and access control—to systems that have been purpose-built and hardened for these tasks over decades.

This simplification has profound implications. It lowers the barrier to entry for building and operating a petabyte-scale lakehouse, making it accessible to organizations whose primary expertise lies in traditional database administration rather than distributed systems engineering. The seamless integration with the broader DuckDB ecosystem, including the MotherDuck managed platform, provides a clear and cost-effective path for users to scale from local analysis on a laptop to a full-fledged cloud data warehouse without changing their tools or workflows.

However, it is crucial to approach DuckLake with a clear understanding of its current state. As an experimental technology, it is still maturing. The ecosystem of tools with first-class support is smaller than that of Iceberg or Delta Lake, and it currently lacks features like declarative constraints that are standard in traditional warehouses. For organizations on the bleeding edge or those whose use cases align perfectly with its strengths—interactive analytics, multi-user collaboration,

and efficient handling of streaming updates—DuckLake presents a compelling and potentially transformative option. For more conservative enterprises or those with deep investments in ecosystems like Apache Spark, a "wait and see" approach may be more prudent, keeping a close watch as the format matures and gains wider adoption.

Ultimately, DuckLake's success will be determined by its ability to build a robust community and foster adoption beyond the DuckDB ecosystem. Its foundation on open standards—SQL and Parquet—positions it well for this future, offering a pragmatic, powerful, and refreshingly simple vision for the future of the data lakehouse.

Works cited

1. DuckLake: SQL as a Lakehouse Format - DuckDB, accessed July 24, 2025, <https://duckdb.org/2025/05/27/ducklake.html>
2. DuckTales: A DuckLake Story. Rethinking the Lakehouse with a ..., accessed July 24, 2025, <https://medium.com/@tfmv/ducktales-a-ducklake-story-3b4d4e400578>
3. DuckLake | MotherDuck Docs, accessed July 24, 2025, <https://motherduck.com/docs/concepts/ducklake/>
4. Quacks & Stacks. DuckLake's One-Table Wonder vs... | by Thomas F McGeehan V | May, 2025 | Medium, accessed July 24, 2025, <https://medium.com/@tfmv/quacks-stacks-5565069a5ef0>
5. A Duck Walks into a Lake - MotherDuck Blog, accessed July 24, 2025, <https://motherduck.com/blog/ducklake-motherduck/>
6. DuckLake in Perspective: Advanced Features and Future Implications - Endjin, accessed July 24, 2025, <https://endjin.com/blog/2025/06/ducklake-perspective-advanced-features-future-implications.html>
7. Getting Started with DuckLake: A New Table Format for Your Lakehouse - MotherDuck Blog, accessed July 24, 2025, <https://motherduck.com/blog/getting-started-ducklake-table-format/>
8. DuckLake is an integrated data lake and catalog format – DuckLake, accessed July 24, 2025, <https://ducklake.select/>
9. DuckLake in Practice: Hands-On Tutorial and Core Features | endjin, accessed July 24, 2025, <https://endjin.com/blog/2025/06/ducklake-practice-hands-on-tutorial-core-features>
10. Get your data ducks in a row with DuckLake | Tigris Object Storage, accessed July 24, 2025, <https://www.tigrisdata.com/blog/ducklake/>
11. Frequently Asked Questions - DuckLake, accessed July 24, 2025, <https://ducklake.select/faq.html>
12. DuckLake + SQLMesh Tutorial: Building a Modern Data Lakehouse, accessed

- July 24, 2025,
<https://www.tobikodata.com/blog/ducklake-sqlmesh-tutorial-a-hands-on>
13. DuckLake - DuckDB, accessed July 24, 2025,
https://duckdb.org/docs/stable/core_extensions/ducklake.html
 14. Connecting - DuckLake, accessed July 24, 2025,
<https://ducklake.select/docs/stable/duckdb/usage/connecting.html>
 15. DuckLake: This is your Data Lake on ACID | Definite, accessed July 24, 2025,
<https://www.definite.app/blog/ducklake>
 16. DuckLake in Perspective: Advanced Features and Future Implications - Endjin, accessed July 24, 2025,
<https://endjin.com/blog/2025/06/ducklake-perspective-advanced-features-future-implications>
 17. DuckLake Privilege Problem : r/DuckDB - Reddit, accessed July 24, 2025,
https://www.reddit.com/r/DuckDB/comments/1l6wqdq/ducklake_privilege_problem/
 18. DuckLake - Sling, accessed July 24, 2025,
<https://docs.slingdata.io/connections/datalake-connections/ducklake>
 19. MotherDuck Managed DuckLakes Now in Preview: Scale to Petabytes, accessed July 24, 2025,
<https://motherduck.com/blog/announcing-ducklake-support-motherduck-preview/>
 20. DuckLake 0.2 - DuckDB, accessed July 24, 2025,
<https://duckdb.org/2025/07/04/ducklake-02.html>
 21. Encryption - DuckLake, accessed July 24, 2025,
https://ducklake.select/docs/stable/duckdb/advanced_features/encryption.html
 22. Data Warehousing Guide | MotherDuck Docs, accessed July 24, 2025,
<https://motherduck.com/docs/getting-started/data-warehouse/>
 23. DuckLake is an integrated data lake and catalog format | Hacker News, accessed July 24, 2025,
<https://news.ycombinator.com/item?id=44106934>
 24. Data Warehousing How-to | MotherDuck Docs, accessed July 24, 2025,
<https://motherduck.com/docs/key-tasks/data-warehousing/>
 25. MotherDuck: DuckDB in the cloud and in the client - CIDR conference, accessed July 24, 2025,
<https://www.cidrdb.org/cidr2024/papers/p46-atwal.pdf>
 26. For App Developers - MotherDuck, accessed July 24, 2025,
<https://motherduck.com/product/app-developers/>
 27. DuckLake : A practical approach to simplify data engineering | by Kaulab Basu - Medium, accessed July 24, 2025,
<https://medium.com/@basu.kaulab/ducklake-a-practical-approach-to-simplify-data-engineering-833384a850a7>
 28. Queries - DuckLake, accessed July 24, 2025,
<https://ducklake.select/docs/stable/specification/queries.html>
 29. Error when migrating from ducklake 0.1 to 0.2 · Issue #240 - GitHub, accessed July 24, 2025,
<https://github.com/duckdb/ducklake/issues/240>

30. Will DuckLake overtake Iceberg? : r/dataengineering - Reddit, accessed July 24, 2025,
https://www.reddit.com/r/dataengineering/comments/1lmmhz4/will_ducklake_overtake_iceberg/
31. Issues · duckdb/ducklake - GitHub, accessed July 24, 2025,
<https://github.com/duckdb/ducklake/issues>
32. MotherDuck SQL | MotherDuck Docs, accessed July 24, 2025,
<https://motherduck.com/docs/sql-reference/motherduck-sql-reference/>
33. Tableau – A Data Visualization Tool - DuckDB, accessed July 24, 2025,
https://duckdb.org/docs/stable/guides/data_viewers/tableau.html
34. Power BI with DuckDB and MotherDuck, accessed July 24, 2025,
<https://motherduck.com/docs/integrations/bi-tools/powerbi/>
35. DuckDB Power Query Custom Connector by MotherDuck - GitHub, accessed July 24, 2025,
<https://github.com/motherduckdb/duckdb-power-query-connector>
36. Looker to DuckDB: Load data with the dlt python library - dltHub, accessed July 24, 2025,
<https://dlthub.com/docs/pipelines/looker/load-data-with-python-from-looker-to-duckdb>
37. ETL vs ELT: Key Differences, Comparisons, & Use Cases - Rivery, accessed July 24, 2025, <https://rivery.io/blog/etl-vs-elt/>
38. The Complete Guide To ELT (Extract, Load, Transform) for Data Workflows - Prophecy, accessed July 24, 2025,
<https://www.prophecy.io/blog/elt-data-pipelines-complete-guide>
39. How to Load Streaming Data into DuckLake with Estuary Flow, accessed July 24, 2025,
<https://estuary.dev/blog/load-streaming-data-ducklake-motherduck/>
40. definite-app/ducklake-speedrun - GitHub, accessed July 24, 2025,
<https://github.com/definite-app/ducklake-speedrun>
41. duckdb/dbt-duckdb: dbt (<http://getdbt.com>) adapter for DuckDB (<http://duckdb.org>) - GitHub, accessed July 24, 2025,
<https://github.com/duckdb/dbt-duckdb>
42. Use DuckDB with Apache Airflow | Astronomer Documentation, accessed July 24, 2025, <https://www.astronomer.io/docs/learn/airflow-duckdb/>
43. AirFlow - MotherDuck, accessed July 24, 2025,
<https://motherduck.com/ecosystem/airflow/>
44. Using DuckDB with Dagster, accessed July 24, 2025,
<https://docs.dagster.io/integrations/libraries/duckdb/using-duckdb-with-dagster>
45. Orchestration | MotherDuck Docs, accessed July 24, 2025,
<https://motherduck.com/docs/integrations/orchestration/>
46. DuckLake & Iceberg: Modern Lakehouse Architecture 2025 | sanj.dev, accessed July 24, 2025,
<https://sanj.dev/post/ducklake-iceberg-modern-lakehouse-architecture-2025>

47. Pricing - MotherDuck, accessed July 24, 2025, <https://motherduck.com/product/pricing/>
48. DuckLake by DuckDB Labs - Data Engineering Blog, accessed July 24, 2025, <https://www.ssp.sh/brain/ducklake/>
49. GDPR vs HIPAA Compliance: What are the Differences? - Securiti.ai, accessed July 24, 2025, <https://securiti.ai/gdpr-vs-hipaa/>
50. GDPR and HIPAA Compliance – Do They Overlap?, accessed July 24, 2025, <https://www.totalhipaa.com/gdpr-and-hipaa/>
51. MotherDuck for Business Analytics: GDPR, SOC 2 Type II, Tiered Support, and New Plan Offerings, accessed July 24, 2025, <https://motherduck.com/blog/introducing-motherduck-for-business-analytics/>
52. Snowflake vs Databricks vs BigQuery - Cloud Data Platform Comparison - Datumo, accessed July 24, 2025, <https://www.datumo.io/blog/snowflake-vs-databricks-vs-bigquery>
53. Databricks vs Snowflake | Performance & Pricing: Comparison Guide - Firebolt, accessed July 24, 2025, <https://www.firebolt.io/comparison/databricks-vs-snowflake>
54. Snowflake vs. BigQuery Comparison: 2025 Deep-Dive - Portable, accessed July 24, 2025, <https://portable.io/learn/snowflake-vs-bigquery-comparison>
55. DuckDB vs. Snowflake vs. Databricks: A Comprehensive Comparison - Sync Computing, accessed July 24, 2025, <https://synccomputing.com/duckdb-snowflake-databricks/>
56. Making an impact with a research project: DuckDB - MotherDuck, accessed July 24, 2025, <https://motherduck.com/videos/61/making-an-impact-with-a-research-project-duckdb/>
57. Digging into Ducklake - rmoff's random ramblings, accessed July 24, 2025, <https://rmoff.net/2025/06/02/digging-into-ducklake/>
58. MotherDuck: Ducking Simple Data Warehouse based on DuckDB, accessed July 24, 2025, <https://motherduck.com/>
59. Case Studies - MotherDuck, accessed July 24, 2025, <https://motherduck.com/case-studies/>
60. duckdb-web/CONTRIBUTING.md at main - GitHub, accessed July 24, 2025, <https://github.com/duckdb/duckdb-web/blob/main/CONTRIBUTING.md>
61. Extensions – DuckDB, accessed July 24, 2025, <https://duckdb.org/docs/extensions/overview>
62. Python Function API - DuckDB, accessed July 24, 2025, <https://duckdb.org/docs/stable/clients/python/function.html>
63. Understanding Custom Functions in DuckDB - DEV Community, accessed July 24, 2025, <https://dev.to/angu10/understanding-custom-functions-in-duckdb-4pml>
64. Customer Support - MotherDuck, accessed July 24, 2025, <https://motherduck.com/customer-support/>
65. Discussions - duckdb ducklake - GitHub, accessed July 24, 2025,

- <https://github.com/duckdb/ducklake/discussions>
66. MotherDuck Videos and Streams, accessed July 24, 2025, <https://motherduck.com/videos/>
67. DuckDB, MotherDuck, Data Engineering, Data Analytics and more - MotherDuck Blog, accessed July 24, 2025, <https://motherduck.com/blog/>
68. Data Lakehouse with dbt and DuckLake - YouTube, accessed July 24, 2025, <https://www.youtube.com/watch?v=zn69Q7FiFfo>